

24
06
26

Practical - 1

Introduction to DevOps Tools & Practices

Aim:-

- Research & document the importance of DevOps in modern software development.
- Install & explore one DevOps tool (e.g., Jenkins, Docker or Ansible).
- Create a diagram showing how CI/CD, monitoring and Infrastructure as Code work together.

Introduction to DevOps Tools & Practices

A. Importance of DevOps in Modern Software Development

What is DevOps?

→ DevOps is a software development approach that combines Development (Dev) & Operations (Ops) teams to improve collaboration, automate workflows, & deliver software faster & more reliably.

Importance of DevOps

- 1) Faster Software Delivery
 - Automates building, testing & deployment processes.
 - Reduces time-to-market for new features & updates.

- 2) Improved Collaboration

- Developers & operations teams work together throughout the software lifecycle.
- Reduces comm. gaps & misunderstandings.

3) Higher Software Quality

- Continuous Testing identifies bugs early.
- Automated quality checks improve reliability.

4) Increased Deployment Frequency

- Org can release updates multiple times a day instead of monthly/quarterly.

5) Better System Reliability

- Continuous monitoring helps detect & fix issues quickly.
- Infrastructure automation reduces config errors.

6) Scalability & Flexibility

- Cloud platforms & Infrastructure as Code (IaC) make scaling easier.

Common DevOps Practices:

- Continuous Integration (CI)
- Continuous Delivery/Deployment (CD)
- Infrastructure as Code (IaC)
- Monitoring & Logging
- Automated Testing
- Version Control (Git)
- Containerization.

Popular DevOps Tools

Category	Tools
Version Control	Git, GitHub, GitLab
CI/CD	Jenkins, GitHub Actions, GitLab CI
Containers	Docker, Podman
Container Orchestration	Kubernetes
Configuration Management	Ansible, Puppet, Chef
Monitoring	Prometheus, Grafana, Nagios

B) Installation & Exploration of Docker

Why Docker?

→ Docker is a containerization platform that packages applications & their dependencies into lightweight containers, ensuring consistent behaviour across environments.

Installation Steps (Windows)

- 1) Visit official Docker Website
- 2) Download Docker Desktop
- 3) Run installer & follow setup wizard.
- 4) Restart system if needed.
- 5) Verify installation.

`docker --version`

Expected output: Docker version XX.X.X

Basic Docker Commands :-

Check Docker Status

→ ~~for~~ docker info

Pull an Image

→ docker pull nginx

View Images

→ docker images

Run a Container

→ docker run -d -p 8080:80 nginx

List Running Containers

⇒ docker ps

Stop a Container

→ docker stop <container-id>

Observations:-

- Docker containers start quickly.
- Apps run consistently across systems.
- Resource usage is lower compared to VM.
- Simplifies deployment & scaling.

Pros of Docker:-

- Portability
- Isolation
- Faster Deployments
- Reduced infrastructure costs.
- Supports microservices architecture.

C) Diagram: How CI/CD, Monitoring & Infrastructure as Code work together

pto →

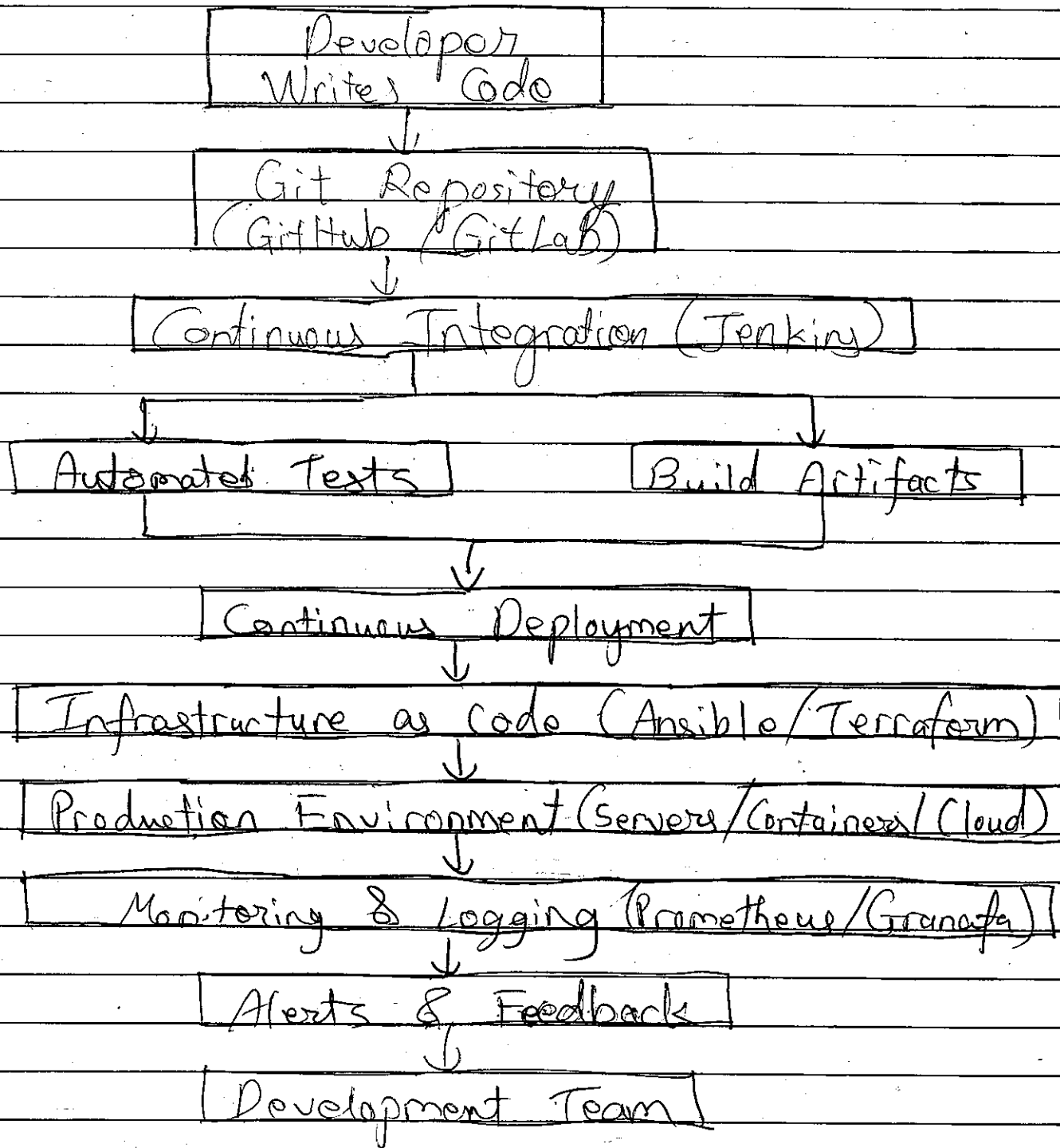
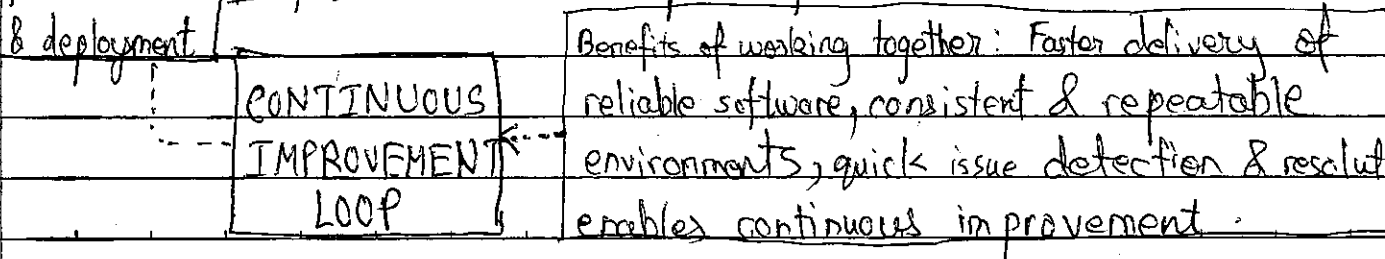
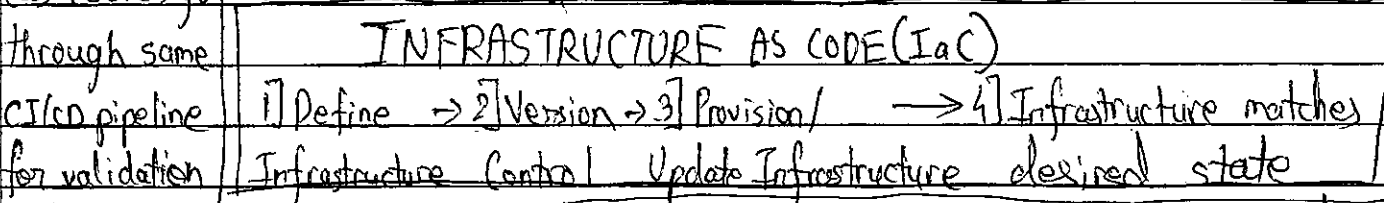
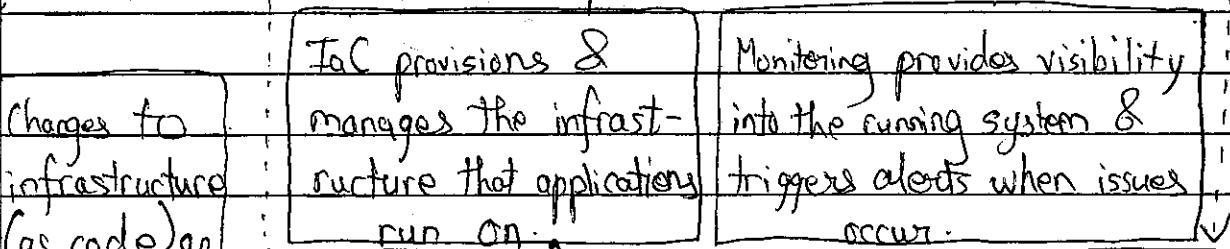
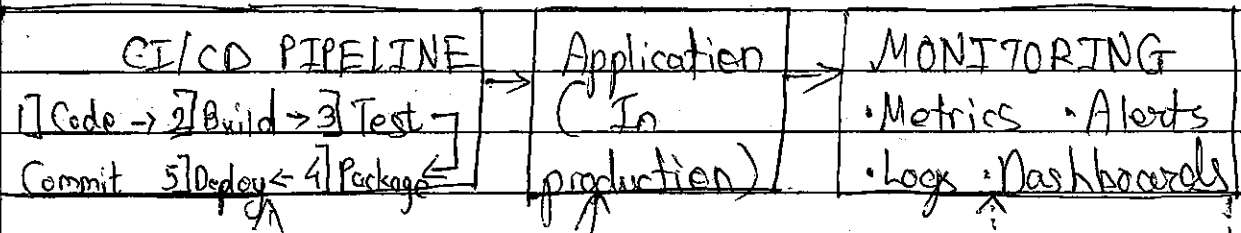


Diagram Explanation

1. Developers commit code to a Git repository.
2. The CI pipeline automatically builds & tests the code.
3. If tests pass, the CD pipeline deploys the application.
4. Infrastructure as Code tools provision & configure servers automatically.

5. The application runs in production.
6. Monitoring tools collect performance metrics & logs.
7. Alerts & feedback help teams identify issues & improve future releases.

Conclusion :- DevOps enables org. to deliver software faster, more reliably & with higher quality. Tools such as Docker, Jenkins, Ansible, Prometheus & Grafana support automation throughout software lifecycle. By integrating CI/CD pipelines, Infrastructure as Code & monitoring systems, teams can achieve continuous improvement & efficient software delivery.



Sr. No.	Date	Topic	Signa
2	30/6/26	Building a REST API with Flask or FastAPI	<u>See</u>

Aim:- Building a REST API with Flask or FastAPI.

- Create a simple REST API using Flask or FastAPI with CRUD operations for a "To-Do List" app.
- Test API using Postman or curl.

→ 1] Check if Python is installed.
In CMD, run: `python --version`
output: Python 3.12.4

2] Create a Project Folder
Create folder for Flask project: `Flask-Project`
Open CMD & go to: `cd path\to\Flask-Project`
Ex:- `cd C:\Users\Nikhita\Docs\Flask-Project`

3] Create a Virtual Environment (Recommended)
Inside project folder, run:
`python -m venv venv`
This created a virtual environment named `venv`.

Activate it:

Windows

`venv\Scripts\activate`

You should see:

`(venv) C:\Users\Nikhita\Docs\Flask-Project >`

4] Install Flask

With virtual environment activated, run:
`pip install flask`

5] Verify Installation

`pip show flask / python -m flask --version`

Ex: output:- Flask 3.x.x Werkzeug 3.x.x

6] Create your first flask app.

Create a file named `app.py`:
`from flask import Flask`

```
app = Flask(__name__)
```

```
@app.route('/')  
def home():
```

```
    return "Hello, Flask!"
```

```
if __name__ == '__main__':  
    app.run(debug = True)
```

7] Run Flask App

In CMD:-

```
python app.py
```

You should see:

* Running on `http://127.0.0.1:5000`

Open your browser & visit

http://127.0.0.1:5000

Output:

Hello, Flask!

Code 2:

Replace code with:

```
from flask import Flask, jsonify
app = Flask(__name__)
```

```
@app.route('/')
def home():
```

```
    return jsonify({
        "message": "Welcome to my first Flask API",
        "status": "success"
    })
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

Run again: python app.py

Visit http://127.0.0.1:5000

Output:

```
{
  "message": "Welcome to my first Flask API",
  "status": "success"
}
```

This is your first REST API endpoint in Flask

Code 3:

```
from flask import Flask, jsonify, request
app = Flask(__name__)
```

```
# Sample data storage
todos = [
```

```
{
    "id": 1,
    "title": "Learn Flask",
    "completed": False
}]
```

```
# CREATE a new task
```

```
@app.route('/todos', methods = ['POST'])
```

```
def create_todo():
```

```
    data = request.get_json()
```

```
    new_todo = {
```

```
        "id": len(todos) + 1,
```

```
        "title": data['title'],
```

```
        "completed": False
    }
```

```
    todos.append(new_todo)
```

```
    return jsonify(new_todo), 201
```

```
# READ all tasks
```

```
@app.route('/todos', methods = ['GET'])
```

```
def get_todos():
```

```
    return jsonify(todos)
```

```
# READ a single task
```

```
@app.route('/todos/<int:id>', methods = ['GET'])
```

```
def get_todo(id):
```

```
    todo = next((t for t in todos if t['id'] == id), None)
```

```
    if todo is None:  
        return jsonify({'message': 'Task not found'}), 404
```

```
    return jsonify(todo)
```

```
# UPDATE a task
```

```
@app.route('/todos/<int:id>', methods=['PUT'])
```

```
def update_todo(id):
```

```
    todo = next((t for t in todos if t['id'] == id), None)
```

```
    if todo is None:  
        return jsonify({'message': 'Task not found'}), 404
```

```
    data = request.get_json()
```

```
    todo['title'] = data.get('title', todo['title'])
```

```
    todo['completed'] = data.get('completed', todo['completed'])
```

```
    return jsonify(todo)
```

```
# DELETE a task
```

```
@app.route('/todos/<int:id>', methods=['DELETE'])
```

```
def delete_todo(id):
```

```
    global todos
```

```
todo = next((t for t in todos if t['id'] == id), None)
```

```
if todo is None:  
    return jsonify({"message": "Task not found"}), 404
```

```
todos = [t for t in todos if t['id'] != id]
```

```
return jsonify({"message": "Task deleted successfully"})
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

Run again: `python app.py`
Visit: `http://127.0.0.1:5000/todos`
`http://127.0.0.1:5000/todos/2`

Output (todos) :-

```
{  
  "completed": false,  
  "id": 1,  
  "title": "Learn Flask"  
}
```

Output (todos/2) :-

```
{  
  "message": "Task not found"  
}
```

```
C:\Users\192>curl http://127.0.0.1:5000/todos  
{"completed": false,  
  "id": 1,  
  "title": "Learn Flask" }
```

```
C:\Users\192>curl http://127.0.0.1:5000/todos/1  
{"completed": false,  
  "id": 1,  
  "title": "Learn Flask" }
```

```
C:\Users\192>curl -X DELETE http://127.0.0.1:5000/todos/2  
{"message": "Task not found" }
```

```
C:\Users\192>curl -X POST http://127.0.0.1:5000/todos -H "Content-Type: application/json" -d '{"title": "Complete REST API Assignment"}'  
{"completed": false,  
  "id": 2,  
  "title": "Complete REST API Assignment" }
```

1. GET all tasks

```
curl http://127.0.0.1:5000/todos
```

```
Output: [{"id": 1,  
          "title": "Learn Flask",  
          "completed": false}]
```

2. POST (Create)

```
curl -X POST http://127.0.0.1:5000/todos -H "Content-Type: application/json" -d '{"title": "Complete REST API Assignment"}'
```

Output: { "id": 1,
"title": "Complete REST API Assignment",
"completed": false }

3. PUT (Update)

curl -X PUT http://127.0.0.1:5000/todos/1 -H "Content-Type: application/json" -d '{"title": "Learn Flask API", "completed": true}'

Output: { "id": 1,
"title": "Learn Flask API",
"completed": true }

4. DELETE

curl -X DELETE http://127.0.0.1:5000/todos/1

Output: { "message": "Task deleted successfully" }

5. VERIFY Deletion

curl http://127.0.0.1:5000/todos

Output: { "id": 2,
"title": "Complete REST API Assignment",
"completed": false }

Sr-No	Date	Topic	Sign
4			

Creating & Managing a VM:

- Set up a VM on cloud provider / locally
- Config networking concepts like VNet
- Deploy Flask/FastAPI app manually on VM

Software Needs:

VMWare Workstation, Ubuntu, Python 3, Flask
Terminal & Browser

(a) Set up VM using VMWare.

1) Open VM Ware

Launch, select Ubuntu VM & Power On.

2) Log in to Ubuntu by username & pass.

3) Update Ubuntu. In terminal, run:

```
sudo apt update
```

```
sudo apt upgrade -y
```

4) Verify Python installation

```
python3 --version
```

If not,

```
sudo apt install python3 python3-pip -y
```

5) Install Flask

```
pip install flask
```

Verify: `pip3 show flask`

Result (a): Ubuntu VM is successfully created

(b) Configure Networking (Virtual Network, IP Address & Ports)

1) Check IP Address:

Run: `ip addr / hostname -I`

2) Understand VM Ware Network Modes

Recommended: Use NAT unless otherwise.
To check or change mode:

i) Shut down VM.

ii) In VMware, select VM.

iii) Go to VM → Settings → Network Adapter

iv) Choose NAT

3) Verify Connection

`ping google.com`

4) Check listening ports:

`sudo ss -tln / netstat -tln`

5) Firewall

`sudo ufw allow 5000`. Then `sudo ufw status`

Result (b)

VM network is configured, IP address is verified & app port is available.

(c) Deploy Flask App manually

1) Create a Project Folder

```
mkdir FlaskApp : cd FlaskApp
```

2) Create an app.py :

```
from flask import Flask
app = Flask(__name__)
app.route('/')
def home():
    return "Hello"
```

```
if __name__ == '__main__':
    app.run(host="0.0.0.0", port=5000)
```

3) Run App :

```
python3 app.py
```

4) Test App :

Using NAT, first test in VM : Open Firefo
in Ubuntu : <http://localhost:5000>

5) Stop Flask Structure
Ctrl + C

Project Structure

```
FlaskApp /
└ app.py
```

Architecture

Host Computer
(Windows, VMware)

|
Ubuntu Virtual Machine

|
Python + Flask App

|
Port 5000 (HTTP)

|
Web Browser

Sr.No.	Date	Topic	Sign
5a	13/7/26	Dockerize a Flask App.	<u>See</u>

Aim: To create a Docker image for Flask app using a Dockerfile

Objective: Create a flask app & needs.txt file
Write a dockerfile, build a Docker image & verify Docker image.

Software Needs

Windows 10/11	→ Any version
Python	→ 3.12+
Docker Desktop	→ Latest
VS Code	→ "
Flask	→ "

A. Write a Dockerfile for your Flask/FastAPI app & build a Docker image.

Step 1: Install Docker Desktop:

Open browser & visit docker.com. Download & Install it. Restart if needed. Open it & wait until Engine status shows Running.

Step 2: Verify Installation:

In CMD, type `docker --version`. Ex. 'Docker version 28.3.2, build xxxx' to be seen. Type 'docker info', & lots of info will be shown.

Step 3 : Create Project Folder :
In CMD, type `mkdir Flask-Project`. Go in folder by
`cd Flask Project`!

Step 4 : Open Folder in VS Code :
Type 'code' & VS Code opens.

Step 5 : Create Flask App.
Create `app.py` file & write the code:

```
from flask import Flask  
app = Flask(__name__)
```

```
@app.route("/")  
def home():  
    return "Welcome to Dockerized Flask Application"
```

```
@app.route("/about")  
def about():  
    return "Docker Practical"
```

```
if __name__ == "__main__":  
    app.run(host = "0.0.0.0", port = 5000)
```

Explanation :

Flask() : creates Flask app

@app.route("/") : Defines Home Page

@app.route("/about") : Defines another page

host = "0.0.0.0" : Allows Docker to access app.

port = 5000 : Flask runs on port 5000

Step 6: Create Virtual Environment (Optional but Recommended)
Type `python -m venv venv` (in CMD) or VS, then `venv\Scripts\activate`. You should see `(venv)` at start of command prompt.

Step 7: Install Flask
`pip install flask`, wait until completion.

Step 8: Test app.
Run `python app.py`. Should see running on `http://127.0.0.1:5000`. Open browser. Check 5000 (should say Welcome) & 5000 /about (should say Docker Practical).

Step 9: Create `needs.txt`
This file has all Python packages needed by app.

Method 1 (recommended):
`pip freeze > needs.txt` &
open file. Should look like:-
`blinker == 1.9.0`
`click == 8.2.1`
`Flask == 3.1.0`

Step 2: Method 2:
Create
`needs.txt`
Write
`flask`

10: In VS Code, you should see:
Flask-Project
├── app.py
├── requirements.txt / needs.txt
└── Dockerfile
 └── venv (optional)

(Verify Project Structure)
[Make sure you're inside Flask Project by running `dir`. You should see `app.py`, `Dockerfile`, `needs.txt`].

Step 11: Open Terminal ↑

12
Step 1: Create Dockerfile
Create a file named 'Dockerfile'. No .txt or extension
In it, paste :-
Step 1: Use Python base image [Downloads Python img. Becomes
FROM python:3.12-slim base OS for container.

Step 2: Set working directory
WORKDIR /app [Creates folder /app in Docker. All files copied here]

Step 3: Copy needs file
COPY needs.txt [Copies needs.txt from PC into Docker container]

Step 4: Install dependencies [Install Flask & other dependencies]
RUN pip install --no-cache-dir -r needs.txt [in Docker in]

Step 5: Copy all project files
COPY [Copies project files in container]

Step 6: Open Flask port
EXPOSE 5000 [Tells Docker that app listens on port 5000]

Step 7: Start app
CMD ["python", "app.py"] [Specifies command Docker runs automatically when container starts]

Step 13: Build Docker Image

Run: docker build -t flask-app

↓ ↓ ↓ ↓
Command on image Assign tag Name of img

Docker will:

- 1) Download Python img (if not already available)
- 2) Copy files
- 3) Install Flask
- 4) Create Docker Image

Ex. Output:

Successfully built 8b1234567890
Successfully tagged flask-app: latest

Step 14: Verify Docker Image

Run: docker images. Ex. output: -

Repository	Tag	Image ID	Size
flask-app	latest	8b1234567890	170MB

Confirms that img. has been built successfully

Expected Result:-

- Docker Desktop is installed & running.
- Flask app works locally.
- Needs.txt is created.
- Dockerfile is created.
- Docker image is successfully built.
- Image appears in output of docker images.

Sr.No	Date	Topic	Sign
5b	13/7/26	Run Docker Container & Test Flask App.	

Aim: To run Docker container for Flask app & verify that app is working correctly.

Ensure that Docker Desktop is installed & running, Flask app is created. Dockerfile is created & Docker image has been built successfully.

Step 1: Open Docker Desktop
Wait until status shows Engine Running

Step 2: Open Project Folder:
Open CMD, Navigate to your project folder:
cd C:\Users\Admin\Docs\Flask Project
Replace path with your actual project folder.

Step 3: Verify Docker Image
Check if img exists: `docker images`. If not, `docker build -t flask-app`.

Step 4: Run Docker (Flask) Container:
`docker run -d -p 5000:5000 flask-app`

Creates & starts new container in detached mode
Runs container on your PC to port 5000 inside container
Maps port 5000 on host to port 5000 inside container
Name of Docker image

Step 5: Verify Container:
docker ps. Status should show Up.

Step 6: Test App.

Open any browser & enter the foll URLs:
http://localhost:5000 | http://localhost:5000/about
Welcome to Dockerized | Docker Practical
Flash App.

Step 7: Test Using Terminal (Optional)

Also do curl http://localhost:5000. Should show
Welcome :-

Step 8: Check Container Logs:

If browser don't display app, view logs:
docker logs <container id>

Ex:- docker logs 4a2d6c8f1b9e

Ex:- output:-

* Running on all addresses (0.0.0.0)

* Running on http://127.0.0.1:5000

The logs help identify startup errors / missing dependencies.

Step 9: Stop container

First, list running containers:

`docker ps`

Then stop container:

`docker stop <container_id>`

Ex:- `docker stop 4a2d6c8f1b9e`

Step 10: Restart container:

List all containers:

`docker ps -a`

Restart stopped container:

`docker start <container_id>`

Ex:- `docker start 4a2d6c8f1b9e`

Open browser again: `http://localhost:5000`

App. should be accessible again

Expected Output:

- Docker container starts successfully.
- Container appears in output of `docker ps`.
- Flask app. opens successfully at `http://localhost:5000`.
- The app. responds correctly to all defined routes.

Sr. No Date
5c 13/7/26

Topic

Sign

Step 1: Open CMD

Run docker images. You must see
REPOSITORY TAG IMAGE ID
flask-app latest a1b2c3d4e5f6
If not; building first

Step 2: log in to Docker Hub

Run: docker login

Enter username & password. If successful,
you'll see: Login Succeeded.

Step 3: Check which acc. is logged in

Run: docker info Look for: Username: yourusername
Make sure it matches your Docker Hub acc.

Step 4: Tag your img

If (ex) Docker Hub username is: nikhita123, Run
docker tag flask-app nikhita123/flask-app:latest

Step 5: Verify Tag

Run: docker images You should see:

REPOSITORY

TAG

flask-app

latest

nikhita123/flask-app

latest

Step 6: Push Image

Run:

```
docker push nikhita129/flask-app:latest
```

If everything is correct, Docker will upload the image.

Step 7: Verify on Docker Hub

- 1) Open <https://hub.docker.com/>
- 2) Log in
- 3) Click Repositories
- 4) You should see flask-app listed.

Sr. No	Date	Topic	Sign
6	20/7/26	Docker Compose for Multi-Container Apps	<u>See</u>

- Create a docker-compose.yml file for a multi-container setup (eg. Flask/Fast API app + PostgreSQL database).
- Run app using Docker Compose & test comms. betⁿ services.

To create & run a multi-container app using Docker Compose with :-

- Flask Application &
- PostgreSQL database

Needs :-

Docker Desktop, VS Code, Python 3.x & Internet

Project Structure :-

FlaskComposeProject/

├── app.py

├── dockerfile

├── docker-compose.yml

├── requirements.txt

└── .env

Step 1: Create Flask App (app.py)

```

from flask import Flask
import psycopg2
import os

```

```
app = Flask(__name__)
```

```
@app.route('/')  
def home():
```

```
    try:
```

```
        conn = psycopg2.connect {  
            host = os.getenv("DB_HOST")  
            database = os.getenv("DB_NAME")  
            user = os.getenv("DB_USER")  
            password = os.getenv("DB_PASSWORD")  
        }
```

```
        conn.close()
```

```
        return "Connected to PostgreSQL successfully!"
```

```
    except Exception as e:
```

```
        return "Database connection failed : " + str(e)
```

```
if __name__ == "__main__":
```

```
    app.run(host="0.0.0.0", port=5000)
```

Step 2: Create requirements.txt

Flask

psycopg2-binary

Step 3: Create Dockerfile

FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt

RUN pip install --no-cache-dir -r requirements.txt

copy.. : EXPOSE 5000 CMD ["python", "app.py"]

Step 4: Create .env file

DB_HOST=db

DB_NAME=mydatabase

DB_USER=postgres

DB_PASSWORD=postgres

Step 5: Create docker-compose.yml

version: "3.9"

services:

web:

build:

container_name: flask-app

ports:

- "5000:5000"

depends_on:

- db

environment:

DB_HOST: db

DB_NAME: mydatabase

DB_USER: postgres

DB_PASSWORD: postgres

db:

image: postgres:16

container_name: postgres_db

restart: always

environment:

POSTGRES_DB: mydatabase

POSTGRES_PASSWORD: postgres

POSTGRES_USER: postgres

ports:

- "5432:5432"

volumes:

- postgres_data: /var/lib/postgresql/data

volumes:

postgres_data:

Part (b): Run the Application Using Docker Compose

Step 1: Open Terminal

Navigate to project folder `cd FlaskComposeProject`

Step 2: Build Containers

`docker compose build`. If successful, Docker builds Flask image.

Step 3: Start Containers

`docker compose up -d`

Expected output:

Creating network "flaskcomposeproject_default".

Step 4: Verify Running Containers

Run `docker ps`. Both containers should be running.

Step 5: Test Comms betⁿ Services

Method 1 (Check Flask logs)

`docker compose logs web`. Should show: Running on

http://0.0.0.0:5000
Method 2 (Recommended)

Open browser. Visit `http://localhost:5000`. Should see: Connected Successfully to PostgreSQL!

Step 6: Stop App.

`docker compose down`. Should see: Stopping flask...

Architecture Diagram:-

